Лабораторная работа №6

Основы микросервисной архитектуры

Цель работы: изучить основные понятия микросервисной архитектуры и создать простейший микросервис.

Теоретическое обоснование

Микросервисы

Архитектурный стиль микросервисов – это подход, при котором единое приложение строится как набор небольших сервисов, каждый из которых работает в собственном процессе и коммуницирует с остальными используя легковесные механизмы, как правило HTTP или gRPC. Эти сервисы построены вокруг бизнес-потребностей и развертываются независимо с использованием полностью автоматизированной среды. Существует абсолютный минимум централизованного управления этими сервисами. Сами по себе эти сервисы могут быть написаны на разных языках и использовать разные технологии хранения данных.

Архитектура микросервисов использует библиотеки, но их основной способ разбиения приложения – путем деления его на сервисы. Мы определяем библиотеки как компоненты, которые подключаются к программе и вызываются ею в том же процессе, в то время как сервисы – это компоненты, выполняемые в отдельном процессе и коммуницирующие между собой через веб-запросы или remote procedure call (RPC).

Go все чаще выбирается для разработки современных распределенных систем. Когда вы разрабатываете облачно-ориентированную распределенную систему, вам может потребоваться поддержка различного специфичного функционала в ваших сервисах, такого как: различные транспортные протоколы (HTTP, gRPC, и др.) и форматы кодирования сообщений для них, надежность RPC, логирование, трассировка, метрики и профилирование, прерывание запросов, ограничение количества запросов, интеграция в

инфраструктуру и даже описание архитектуры. Go популярный язык благодаря своей простоте и подходам "без магии", поэтому пакеты Go, например, стандартная библиотека, уже подходят для разработки распределенных систем больше, чем использование полноценного фреймворка с множеством "магии под капотом".

Введение в Go kit

Go kit – это набор пакетов Go, которые облегчают создание надежных и поддерживаемых микросервисов. Go kit предоставляет библиотеки для реализации различных компонентов прозрачной и надежной архитектуры приложения, используя такие слои как: логирование, метрики, трассировка, ограничение и прерывание запросов, которые необходимы для запуска микросервисов на проде. Go kit хорош тем, что в нем хорошо реализованы инструменты для взаимодействия с различными инфраструктурами, форматами кодирования сообщений и различными транспортными уровнями.

Архитектура Go kit

Три главных уровня в архитектуре приложения разработанных с помощью Go kit это:

1. Транспортный уровень

Привязка сервиса к конкретному протоколу (HTTP, gRPC, NATS). Один сервис может одновременно предоставляться через несколько транспортов.

Когда вы пишите микросервисы для распределенных систем, сервисам в них часто приходится общаться друг с другом, используя различные транспортные протоколы, такие как: HTTP или gRPC, или использовать pub/sub системы, например NATS. Транспортный уровень в Go kit привязывается к конкретному транспортному протоколу (далее транспорт). Go kit поддерживает различный транспорт для работы вашего сервиса, такой как: HTTP, gRPC, NATS, AMQP и Thirft (также вы можете разработать свой транспорт под свой протокол). Поэтому сервисы, написанные с помощью Go kit, часто акцентируют внимание на реализации конкретной бизнес-логики, которая ничего не знает о используемом транспорте, вы свободны

использовать различные транспорты для одного и того же сервиса. Как пример, один сервис, написанный на Go kit может одновременно предоставлять доступ к нему по HTTP и gRPC.

2. Уровень эндпоинтов

Каждый метод сервиса преобразуется в endpoint.Endpoint, который представляет собой отдельный RPC-метод. Эндпоинт – это функция вида:

```
type Endpoint func(ctx context.Context, request interface{}) (response interface{}, err error)
```

Конечная точка или эндпоинт – это фундаментальный «строительный кирпичик» для сервисов и клиентов. В Go kit основной паттерн общения – это RPC. Эндпоинт представляется как отдельный RPC метод. Каждый метод сервиса в Go kit преобразуется в эндпоинт, позволяющий общаться между сервером и клиентом в RCP стиле. Каждый эндпоинт выставляет наружу сервиса метод, используя Транспортный уровень, который в свою очередь использует различные транспортные протоколы, например HTTP или gRPC. Отдельный эндпоинт может выставляться наружу сервиса одновременно с помощью нескольких транспортов (HTTP и gRPC на разных портах).

3. Уровень сервиса

Бизнес-логика, спроектированная как интерфейс. Сервис не знает ничего о транспорте, эндпоинтах и кодировании сообщений. Это позволяет придерживаться чистой архитектуры.

Бизнес логика реализуется в сервисном слое. Сервисы, написанные с Go kit, проектируются как интерфейсы. Бизнес логика в сервисном слое содержит основное ядро бизнес-логики, которая не должна знать ничего о используемых эндпоинтах или конкретном транспортном протоколе, как HTTP или gRPC, или о кодировании или декодировании запросов и ответов различных типов сообщений. Это позволит вам придерживаться чистой архитектуры в сервисах, написанных с помощью Go kit. Каждый метод сервиса преобразуется в эндпоинт с помощью адаптера и выставляется наружу с помощью конкретного

транспорта. Благодаря использованию чистой архитектуры, отдельный метод может быть выставлен с помощью нескольких транспортов одновременно.

Схема взаимодействия:

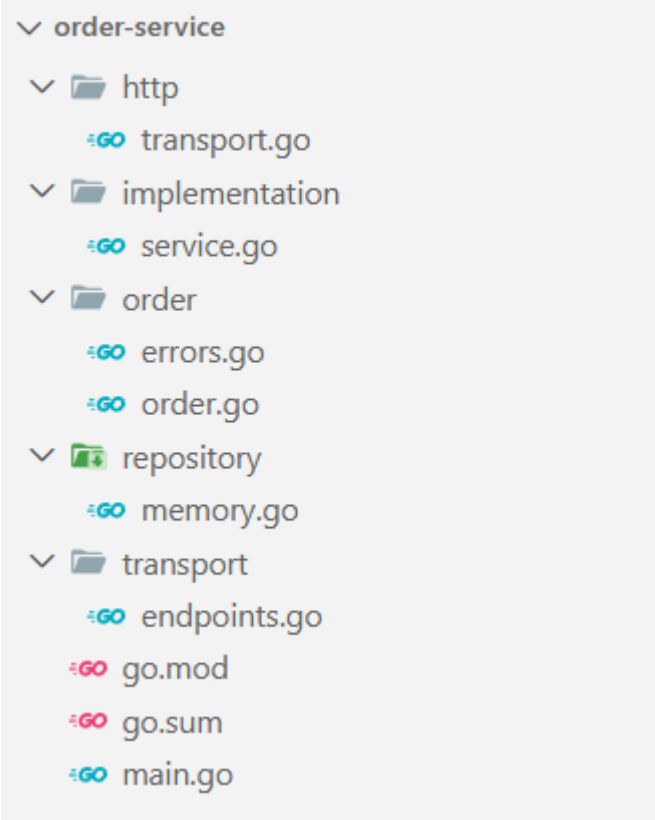
HTTP-запрос → Транспорт (decode) → Endpoint → Service → Repository

HTTP-ответ ← Транспорт (encode) ← Endpoint ← Service ← Repository

Примеры

Рассмотрим создание микросервиса для управления заказами в системе электронной коммерции. В примере используется in-memory хранилище (map), чтобы сосредоточиться на архитектуре Go kit, а не на настройке базы данных.

Структура проекта:



Пояснение к структуре:

order.go – сущности и интерфейсы

errors.go – ошибки сервиса

service.go – реализация сервисного слоя

memory.go – in-memory реализация Repository

endpoints.go – MakeEndpoints

transport.go – HTTP-транспорт

Шаг 1. Инициализация модуля

Создаём рабочий каталог и инициализируем Go-модуль. Модуль – это единица версионирования и управления зависимостями в Go (начиная с версии 1.11).

bash

```
mkdir order-service
cd order-service
go mod init example.com/order-service
go get github.com/go-kit/kit
go get github.com/google/uuid
go get github.com/gorilla/mux
```

После выполнения команд появится файл go.mod с описанием модуля и его зависимостей, а также go.sum с контрольными суммами.

Шаг 2. Бизнес-логика в сервисе: сущности предметной области (order/order.go)

Определяем сущности предметной области Order и OrderItem, а также интерфейсы Service и Repository.

В Go kit бизнес-логика проектируется как интерфейс. Это ключевой принцип чистой архитектуры: сервисный слой не должен знать ничего о транспорте (HTTP или gRPC) и о том, как хранятся данные. Интерфейс Service описывает что делает сервис, а не как. Интерфейс Repository отделяет бизнес-логику от хранилища – это позволяет заменить in-memory хранилище на PostgreSQL или CockroachDB без изменения сервисного слоя.

```
package order

import "context"

// Order представляет заказ
type Order struct {
    ID           string      `json:"id,omitempty"`
    CustomerID   string      `json:"customer_id"`
    Status       string      `json:"status"`
    CreatedOn    int64       `json:"created_on,omitempty"`
    RestaurantID string      `json:"restaurant_id"`
    OrderItems   []OrderItem `json:"order_items,omitempty"`
}

// OrderItem представляет позицию заказа
type OrderItem struct {
    ProductCode string  `json:"product_code"`
```

```
        Name        string  `json:"name"`
        UnitPrice   float32 `json:"unit_price"`
        Quantity    int32   `json:"quantity"`
    }

    // Service описывает бизнес-логику сервиса заказов
    type Service interface {
        Create(ctx context.Context, order Order) (string, error)
        GetByID(ctx context.Context, id string) (Order, error)
        ChangeStatus(ctx context.Context, id string, status string)
error
    }

    // Repository описывает хранилище заказов
    type Repository interface {
        CreateOrder(ctx context.Context, order Order) error
        GetOrderByID(ctx context.Context, id string) (Order, error)
        ChangeOrderStatus(ctx context.Context, id string, status
string) error
    }
```

Пояснения к коду:

1. ctx context.Context — стандартный способ передачи контекста (таймауты, отмена) в Go.

2. Теги json:"..." управляют сериализацией в JSON. omitempty означает, что поле не попадёт в JSON, если оно пустое.

3. Метод Create возвращает string (ID созданного заказа) и error.

4. Метод GetByID возвращает сущность и ошибку.

5. ChangeStatus возвращает только ошибку – это команда без результата.

Шаг 3. Ошибки сервиса (order/errors.go)

Определяем набор доменных ошибок.

В Go ошибки – это значения. Вместо того чтобы возвращать строки, мы определяем именованные ошибки. Это позволяет вызывающему коду проверять ошибки через errors.Is() и не зависеть от текста сообщения.

```
    package order


    import "errors"
```

```
var (
    ErrOrderNotFound   = errors.New("order not found")
    ErrCmdRepository   = errors.New("unable to command repository")
    ErrQueryRepository = errors.New("unable to query repository")
)
```

Шаг 4. Реализация сервисного слоя (implementation/service.go)

Реализуем интерфейс Service.

Сервисный слой – это ядро бизнес-логики. Он ничего не знает про HTTP или gRPC, не занимается сериализацией. Здесь только бизнес-правила: генерация ID, установка статуса «Pending» по умолчанию, установка времени создания. Логирование вынесено в отдельный слой (go-kit/log), что позволяет легко заменить логгер или добавить метрики.

```
package implementation

import (
    "context"
    "time"

    "github.com/go-kit/kit/log"
    "github.com/go-kit/kit/log/level"
    "github.com/google/uuid"

    "example.com/order-service/order"
)

type service struct {
    repository order.Repository
    logger     log.Logger
}

// NewService создаёт новый экземпляр сервиса заказов
```

```
    func NewService(repo order.Repository, logger log.Logger) order.Service {
        return &service{
            repository: repo,
            logger:     logger,
        }
    }

    func (s *service) Create(ctx context.Context, o order.Order) (string, error) {
        logger := log.With(s.logger, "method", "Create")

        id := uuid.New().String()
        o.ID = id
        o.Status = "Pending"
        o.CreatedOn = time.Now().Unix()

        if err := s.repository.CreateOrder(ctx, o); err != nil {
            level.Error(logger).Log("err", err)
            return "", order.ErrCmdRepository
        }
        return id, nil
    }

    func (s *service) GetByID(ctx context.Context, id string) (order.Order, error) {
        logger := log.With(s.logger, "method", "GetByID")

        o, err := s.repository.GetOrderByID(ctx, id)
        if err != nil {
            level.Error(logger).Log("err", err)
            return o, order.ErrOrderNotFound
        }
        return o, nil
```

```
    }

    func (s *service) ChangeStatus(ctx context.Context, id string,
status string) error {
        logger := log.With(s.logger, "method", "ChangeStatus")

        if err := s.repository.ChangeOrderStatus(ctx, id, status); err
!= nil {
            level.Error(logger).Log("err", err)
            return order.ErrCmdRepository
        }
        return nil
    }
```

Пояснения к коду:

1. service — неэкспортируемая структура. Создать её можно только через NewService – это паттерн «конструктор».

2. log.With(s.logger, "method", "Create") добавляет к каждой записи лога поле method, чтобы было понятно, какой метод логирует.

3. uuid.New().String() генерирует уникальный идентификатор заказа.

4. Бизнес-правила: при создании заказа статус всегда Pending, время создания – текущее Unix-время.

Шаг 5. In-memory репозиторий (repository/memory.go)

Реализуем интерфейс Repository через map[string]Order.

Для учебной работы нет необходимости поднимать полноценную СУБД. In-memory репозиторий позволяет сосредоточиться на архитектуре Go kit. При этом интерфейс Repository гарантирует, что замена на реальную БД не потребует изменений в сервисном слое.

Используем sync.RWMutex для безопасного доступа из нескольких горутин: HTTP-сервер обрабатывает запросы параллельно, и без блокировки возможны гонки данных.

```
package repository

import (
    "context"
    "sync"

    "example.com/order-service/order"
)

type memoryRepo struct {
    mu     sync.RWMutex
    orders map[string]order.Order
}

// NewMemoryRepository создаёт in-memory хранилище заказов
func NewMemoryRepository() order.Repository {
    return &memoryRepo{
        orders: make(map[string]order.Order),
    }
}

func (r *memoryRepo) CreateOrder(ctx context.Context, o order.Order) error {
    r.mu.Lock()
    defer r.mu.Unlock()
    r.orders[o.ID] = o
    return nil
}

func (r *memoryRepo) GetOrderByID(ctx context.Context, id string) (order.Order, error) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    o, ok := r.orders[id]
```

```
        if !ok {
            return order.Order{}, order.ErrOrderNotFound
        }
        return o, nil
    }

    func (r *memoryRepo) ChangeOrderStatus(ctx context.Context, id,
status string) error {
        r.mu.Lock()
        defer r.mu.Unlock()
        o, ok := r.orders[id]
        if !ok {
            return order.ErrOrderNotFound
        }
        o.Status = status
        r.orders[id] = o
        return nil
    }
```

Пояснения к коду:

1. RWMutex позволяет нескольким читателям работать одновременно (RLock), но запись блокирует всех (Lock).

2. defer r.mu.Unlock() – отложенная разблокировка, сработает при выходе из функции даже при панике.

3. o, ok := r.orders[id] – идиоматическая проверка наличия ключа в map. Если ok == false, заказ не найден.

Шаг 6. Запросы и ответы для RPC-эндпоинтов (transport/endpoints.go)

Определяем DTO (Data Transfer Object) для запросов и ответов, а также создаём эндпоинты Go kit.

Методы сервиса – это Go-функции. Но клиенты (другие сервисы, фронтенд) общаются с сервисом через сеть, и им нужны структуры, которые можно сериализовать (JSON, protobuf). DTO – это отдельные структуры, которые описывают, что приходит и что уходит по сети. Они отделены от доменных сущностей, чтобы изменения в API не ломали бизнес-логику.

Каждый метод сервиса преобразуется в эндпоинт с помощью адаптера. Адаптер:

1. Принимает interface{} (обобщённый запрос).

2. Приводит его к конкретному типу через type assertion (request.(CreateRequest)).

3. Вызывает соответствующий метод сервиса.

4. Оборачивает результат в DTO-ответ.

```
package transport

import (
    "context"

    "github.com/go-kit/kit/endpoint"

    "example.com/order-service/order"
)

// DTO для запросов и ответов
type CreateRequest struct {
    Order order.Order
}

type CreateResponse struct {
    ID  string `json:"id"`
    Err error  `json:"error,omitempty"`
}

type GetByIDRequest struct {
```

```
    ID string `json:"id"`
}

type GetByIDResponse struct {
    Order order.Order `json:"order"`
    Err   error       `json:"error,omitempty"`
}

type ChangeStatusRequest struct {
    ID     string `json:"id"`
    Status string `json:"status"`
}

type ChangeStatusResponse struct {
    Err error `json:"error,omitempty"`
}

// Endpoints — набор эндпоинтов сервиса заказов
type Endpoints struct {
    Create       endpoint.Endpoint
    GetByID      endpoint.Endpoint
    ChangeStatus endpoint.Endpoint
}

// MakeEndpoints создаёт эндпоинты для сервиса заказов
func MakeEndpoints(s order.Service) Endpoints {
    return Endpoints{
        Create:       makeCreateEndpoint(s),
        GetByID:      makeGetByIDEndpoint(s),
        ChangeStatus: makeChangeStatusEndpoint(s),
    }
}

func makeCreateEndpoint(s order.Service) endpoint.Endpoint {
```

```
        return func(ctx context.Context, request interface{})
(interface{}, error) {
            req := request.(CreateRequest)
            id, err := s.Create(ctx, req.Order)
            return CreateResponse{ID: id, Err: err}, nil
        }
    }

    func makeGetByIDEndpoint(s order.Service) endpoint.Endpoint {
        return func(ctx context.Context, request interface{})
(interface{}, error) {
            req := request.(GetByIDRequest)
            o, err := s.GetByID(ctx, req.ID)
            return GetByIDResponse{Order: o, Err: err}, nil
        }
    }

    func makeChangeStatusEndpoint(s order.Service) endpoint.Endpoint {
        return func(ctx context.Context, request interface{})
(interface{}, error) {
            req := request.(ChangeStatusRequest)
            err := s.ChangeStatus(ctx, req.ID, req.Status)
            return ChangeStatusResponse{Err: err}, nil
        }
    }
```

Пояснения к коду:

1. request.(CreateRequest) — type assertion. Если тип не совпадёт, будет паника; поэтому транспортный уровень обязан передать запрос правильного типа.

2. Адаптер возвращает nil в качестве второй ошибки, потому что бизнес-ошибка уже упакована в DTO-ответ (CreateResponse.Err). Это позволяет клиенту видеть структурированный ответ, а не общую HTTP-ошибку.

3. Один и тот же эндпоинт может быть выставлен через разные транспорты одновременно (HTTP, gRPC).

Шаг 7. Выставление сервиса наружу с помощью HTTP (http/transport.go)

Привязываем эндпоинты Go kit к HTTP-транспорту.

Эндпоинты – это абстракция, не привязанная к протоколу. Транспортный уровень отвечает за:

– маршрутизацию (gorilla/mux);

– декодирование HTTP-запроса в DTO (DecodeRequestFunc);

– кодирование DTO в HTTP-ответ (EncodeResponseFunc);

– обработку ошибок (ServerErrorEncoder).

Функция kithttp.NewServer из пакета github.com/go-kit/kit/transport/http создаёт http.Handler, который оборачивает эндпоинт и вызывает функции кодирования/декодирования.

```
package http

import (
    "context"
    "encoding/json"
    "net/http"

    "github.com/go-kit/kit/log"
    kithttp "github.com/go-kit/kit/transport/http"
    "github.com/gorilla/mux"

    "example.com/order-service/transport"
)

// NewService связывает эндпоинты Go kit с HTTP-транспортом
func NewService(endpoints transport.Endpoints, logger log.Logger) http.Handler {
```

```
        r := mux.NewRouter()

        options := []kithttp.ServerOption{
            kithttp.ServerErrorLogger(logger),
            kithttp.ServerErrorEncoder(encodeError),
        }

        r.Methods("POST").Path("/orders").Handler(kithttp.NewServer(
            endpoints.Create,
            decodeCreateRequest,
            encodeResponse,
            options...,
        ))


r.Methods("GET").Path("/orders/{id}").Handler(kithttp.NewServer(
            endpoints.GetByID,
            decodeGetByIDRequest,
            encodeResponse,
            options...,
        ))


r.Methods("PATCH").Path("/orders/{id}/status").Handler(kithttp.NewServ
er(
            endpoints.ChangeStatus,
            decodeChangeStatusRequest,
            encodeResponse,
            options...,
        ))

        return r
    }
```

```
    func decodeCreateRequest(_ context.Context, r *http.Request)
(interface{}, error) {
        var req transport.CreateRequest
        if err := json.NewDecoder(r.Body).Decode(&req.Order); err !=
nil {
            return nil, err
        }
        return req, nil
    }

    func decodeGetByIDRequest(_ context.Context, r *http.Request)
(interface{}, error) {
        vars := mux.Vars(r)
        return transport.GetByIDRequest{ID: vars["id"]}, nil
    }

    func decodeChangeStatusRequest(_ context.Context, r *http.Request)
(interface{}, error) {
        vars := mux.Vars(r)
        var body struct {
            Status string `json:"status"`
        }
        if err := json.NewDecoder(r.Body).Decode(&body); err != nil {
            return nil, err
        }
        return transport.ChangeStatusRequest{ID: vars["id"], Status:
body.Status}, nil
    }

    func encodeResponse(ctx context.Context, w http.ResponseWriter,
response interface{}) error {
        w.Header().Set("Content-Type", "application/json; charset=utf-
8")
        return json.NewEncoder(w).Encode(response)
```

```
}

func encodeError(_ context.Context, err error, w
http.ResponseWriter) {
    w.Header().Set("Content-Type", "application/json; charset=utf-
8")
    w.WriteHeader(http.StatusInternalServerError)
    json.NewEncoder(w).Encode(map[string]interface{}{"error":
err.Error()})
}
```

Пояснения к коду:

1. kithttp.NewServer(endpoint, decoder, encoder, options...) — создаёт HTTP-обработчик для одного эндпоинта.

2. DecodeRequestFunc имеет сигнатуру func(context.Context, *http.Request) (interface{}, error).

3. EncodeResponseFunc имеет сигнатуру func(context.Context, http.ResponseWriter, interface{}) error.

4. ServerErrorEncoder вызывается, если декодер или эндпоинт вернули ошибку.

5. Маршруты: POST /orders (создание), GET /orders/{id} (получение), PATCH /orders/{id}/status (смена статуса).

Шаг 8. Запуск HTTP-сервера (main.go)

Собираем все слои вместе и запускаем HTTP-сервер.

В main.go выполняется «склейка» приложения:

1. Настраивается логгер.

2. Создаётся репозиторий (repository.NewMemoryRepository).

3. Создаётся сервис (implementation.NewService).

4. Создаются эндпоинты (transport.MakeEndpoints).

5. Создаётся HTTP-обработчик (http.NewService).

6. Запускается сервер.

Также реализован graceful shutdown – при получении сигнала SIGINT (Ctrl+C) или SIGTERM сервер корректно завершает работу.

```
package main

import (
    "flag"
    "fmt"
    "net/http"
    "os"
    "os/signal"
    "syscall"

    "github.com/go-kit/kit/log"
    "github.com/go-kit/kit/log/level"

    "example.com/order-service/http"
    "example.com/order-service/implementation"
    "example.com/order-service/repository"
    "example.com/order-service/transport"
)

func main() {
    httpAddr := flag.String("http.addr", ":8080", "HTTP listen address")
    flag.Parse()

    // Настройка логгера
    var logger log.Logger
    {
        logger = log.NewLogfmtLogger(os.Stderr)
        logger = log.NewSyncLogger(logger)
        logger = level.NewFilter(logger, level.AllowDebug())
        logger = log.With(logger,
```

```
            "svc", "order",
            "ts", log.DefaultTimestampUTC,
            "caller", log.DefaultCaller,
        )
    }

    level.Info(logger).Log("msg", "service started")
    defer level.Info(logger).Log("msg", "service ended")

    // Создание сервиса
    repo := repository.NewMemoryRepository()
    svc := implementation.NewService(repo, logger)

    // Создание HTTP-обработчика
    endpoints := transport.MakeEndpoints(svc)
    handler := http.NewService(endpoints, logger)

    // Запуск сервера
    errs := make(chan error)
    go func() {
        c := make(chan os.Signal, 1)
        signal.Notify(c, syscall.SIGINT, syscall.SIGTERM)
        errs <- fmt.Errorf("%s", <-c)
    }()

    go func() {
        level.Info(logger).Log("transport",    "HTTP",    "addr",
*httpAddr)
        server := &http.Server{
            Addr:    *httpAddr,
            Handler: handler,
        }
        errs <- server.ListenAndServe()
    }()
```

```
    level.Error(logger).Log("exit", <-errs)
}
```

Пояснения к коду:

1. flag.String("http.addr", ":8080", ...) – позволяет задать адрес через флаг -http.addr=:9090.

2. log.NewSyncLogger – обёртка, которая делает запись в лог потокобезопасной.

3. level.NewFilter(logger, level.AllowDebug()) – фильтр уровня логирования.

4. log.With(...) – добавляет постоянные поля к каждой записи (имя сервиса, время, вызывающий).

5. errs := make(chan error) — канал для сбора ошибок из горутин.

6. Graceful shutdown: одна горутина слушает сигналы ОС, другая – запускает сервер. <-errs блокирует main, пока не придёт сигнал или ошибка.

Шаг 9. Запуск и проверка

Bash	`go run main.go`

Проверка работы через curl:

Bash

```
# Создать заказ
curl -X POST http://localhost:8080/orders \
  -H "Content-Type: application/json" \
  -d '{
    "customer_id": "cust-1",
    "restaurant_id": "rest-42",
    "order_items": [
      {"product_code": "P001", "name": "Пицца", "unit_price":
550.0, "quantity": 1}
    ]
  }'
# Ответ: {"id":"<uuid>"}

# Получить заказ по ID
curl http://localhost:8080/orders/<uuid>

# Изменить статус
curl -X PATCH http://localhost:8080/orders/<uuid>/status \
  -H "Content-Type: application/json" \
  -d '{"status":"Completed"}'
```

Задания

Задание 1. Базовое

Повторить приведённый пример:

1. Создать структуру проекта.

2. Собрать и запустить сервис.

3. Проверить все три эндпоинта через curl.

4. Убедиться, что данные сохраняются между запросами (в памяти).

Для корректной работы сервиса достаточно запустить файл с сервером, через go run. Для проверки работоспособности следует перейти по адресу localhost:8080/orders/1.

Задание 2. Свой сервис по варианту

Реализовать микросервис по своему варианту (см. раздел «Варианты индивидуальных заданий»), используя ту же трёхслойную архитектуру Go kit:

– сервисный слой (интерфейс + реализация);

– эндпоинты;

– HTTP-транспорт;

– in-memory репозиторий.

Задание 3. Второй транспорт – gRPC

Выставить тот же сервис наружу одновременно через HTTP и gRPC:

– HTTP на порту :8080;

– gRPC на порту :9090.

Для этого:

1. Описать .proto-файл с сервисом и сообщениями.
2. Сгенерировать Go-код через protoc.
3. Реализовать gRPC-транспорт через github.com/go-kit/kit/transport/grpc.
4. Запустить оба транспорта в main.go параллельно.
5. Проверить через grpcurl.

Варианты индивидуальных заданий

Вариант 1. Сервис «Библиотека»

Сущность Book: ID, Title, Author, Year int, ISBN, Available bool.

Методы сервиса:

Create(ctx, book Book) (string, error)

GetByID(ctx, id string) (Book, error)

UpdateAuthor(ctx, id string, author string) error

Вариант 2. Сервис «Задачи» (Todo)

Сущность Task: ID, Title, Description, Done bool, CreatedOn int64, Priority int.

Методы сервиса:

Create(ctx, task Task) (string, error)

GetByID(ctx, id string) (Task, error)

MarkDone(ctx, id string) error

Вариант 3. Сервис «Студенты»

Сущность Student: ID, FullName, Group, GPA float32, EnrolledOn int64.

Методы сервиса:

Create(ctx, student Student) (string, error)

GetByID(ctx, id string) (Student, error)

ChangeGroup(ctx, id string, group string) error

Вариант 4. Сервис «Платежи»

Сущность Payment: ID, Amount float64, Currency string, Status string, CreatedOn int64, SenderID string.

Методы сервиса:

Create(ctx, payment Payment) (string, error)

GetByID(ctx, id string) (Payment, error)

ChangeStatus(ctx, id string, status string) error

Вариант 5. Сервис «Рестораны»

Сущность Restaurant: ID, Name, Address, Rating float32, Open bool.

Методы сервиса:

Create(ctx, restaurant Restaurant) (string, error)

GetByID(ctx, id string) (Restaurant, error)

ChangeRating(ctx, id string, rating float32) error

Вариант 6. Сервис «Сотрудники»

Сущность Employee: ID, FullName, Position, Salary float64, HiredOn int64.

Методы сервиса:

Create(ctx, employee Employee) (string, error)

GetByID(ctx, id string) (Employee, error)

ChangePosition(ctx, id string, position string) error

Вариант 7. Сервис «Курсы»

Сущность Course: ID, Title, Instructor, Duration int (часы), Published bool.

Методы сервиса:

Create(ctx, course Course) (string, error)

GetByID(ctx, id string) (Course, error)

Publish(ctx, id string) error

Вариант 8. Сервис «Билеты»

Сущность Ticket: ID, EventName, Seat, Price float64, Sold bool.

Методы сервиса:

Create(ctx, ticket Ticket) (string, error)

GetByID(ctx, id string) (Ticket, error)

Sell(ctx, id string) error

Вариант 9. Сервис «Книжный магазин»

Сущность Book: ID, Title, Author, Price float64, Stock int, InStock bool.

Методы сервиса:

Create(ctx, book Book) (string, error)

GetByID(ctx, id string) (Book, error)

UpdateStock(ctx, id string, delta int) error

Вариант 10. Сервис «Аптека»

Сущность Medicine: ID, Name, Manufacturer, Price float64, Prescription bool, Quantity int.

Методы сервиса:

Create(ctx, medicine Medicine) (string, error)

GetByID(ctx, id string) (Medicine, error)

Sell(ctx, id string, count int) error

Вариант 11. Сервис «Автомобили»

Сущность Car: ID, Brand, Model, Year int, Price float64, Sold bool.

Методы сервиса:

Create(ctx, car Car) (string, error)

GetByID(ctx, id string) (Car, error)

MarkSold(ctx, id string) error

Вариант 12. Сервис «Отели»

Сущность Room: ID, HotelName, RoomNumber, PricePerNight float64, Occupied bool, Capacity int.

Методы сервиса:

Create(ctx, room Room) (string, error)

GetByID(ctx, id string) (Room, error)

CheckIn(ctx, id string) error

Вариант 13. Сервис «Авиарейсы»

Сущность Flight: ID, FlightNumber, Departure, Arrival, DepartureTime int64, AvailableSeats int.

Методы сервиса:

Create(ctx, flight Flight) (string, error)

GetByID(ctx, id string) (Flight, error)

BookSeat(ctx, id string) error

Вариант 14. Сервис «Склад»

Сущность Product: ID, SKU, Name, Quantity int, Price float64, Location string.

Методы сервиса:

Create(ctx, product Product) (string, error)

GetByID(ctx, id string) (Product, error)

MoveToLocation(ctx, id string, location string) error

Вариант 15. Сервис «Счета»

Сущность Invoice: ID, CustomerID, Amount float64, Currency string, IssuedOn int64, Paid bool.

Методы сервиса:

Create(ctx, invoice Invoice) (string, error)

GetByID(ctx, id string) (Invoice, error)

MarkPaid(ctx, id string) error

Вариант 16. Сервис «Подписки»

Сущность Subscription: ID, UserID, Plan string, Price float64, StartDate int64, Active bool.

Методы сервиса:

Create(ctx, sub Subscription) (string, error)

GetByID(ctx, id string) (Subscription, error)

Cancel(ctx, id string) error

Вариант 17. Сервис «Мероприятия»

Сущность Event: ID, Title, Venue, EventDate int64, TicketPrice float64, TicketsLeft int.

Методы сервиса:

Create(ctx, event Event) (string, error)

GetByID(ctx, id string) (Event, error)

BuyTicket(ctx, id string) error

Вариант 18. Сервис «Кинобилеты»

Сущность MovieTicket: ID, MovieTitle, Hall string, Seat string, ShowTime int64, Sold bool.

Методы сервиса:

Create(ctx, ticket MovieTicket) (string, error)

GetByID(ctx, id string) (MovieTicket, error)

Sell(ctx, id string) error

Вариант 19. Сервис «Рецепты»

Сущность Recipe: ID, Title, Author, CookingTimeMin int, Servings int, Published bool.

Методы сервиса:

Create(ctx, recipe Recipe) (string, error)

GetByID(ctx, id string) (Recipe, error)

Publish(ctx, id string) error

Вариант 20. Сервис «Доставка»

Сущность Delivery: ID, OrderID, CourierID, Address, Status string, CreatedOn int64.

Методы сервиса:

Create(ctx, delivery Delivery) (string, error)

GetByID(ctx, id string) (Delivery, error)

ChangeStatus(ctx, id string, status string) error

Сводная таблица вариантов

№	Домен	Сущность	Доменный метод
1	Библиотека	Book	UpdateAuthor
2	Задачи	Task	MarkDone
3	Студенты	Student	ChangeGroup
4	Платежи	Payment	ChangeStatus
5	Рестораны	Restaurant	ChangeRating
6	Сотрудники	Employee	ChangePosition
7	Курсы	Course	Publish
8	Билеты	Ticket	Sell
9	Книжный магазин	Book	UpdateStock
10	Аптека	Medicine	Sell
11	Автомобили	Car	MarkSold
12	Отели	Room	CheckIn
13	Авиарейсы	Flight	BookSeat
14	Склад	Product	MoveToLocation
15	Счета	Invoice	MarkPaid
16	Подписки	Subscription	Cancel
17	Мероприятия	Event	BuyTicket
18	Кинобилеты	MovieTicket	Sell
19	Рецепты	Recipe	Publish
20	Доставка	Delivery	ChangeStatus